

Department of Computer Science

University of Engineering and Technology Lahore
Pakistan

هم نوا

Hamnawa

Accessible Document Reader for Visually Impaired Students

Design Architecture Document

Subject: Software Engineering **Group No.:** 2

Submitted by:

Muhammad Sarim	2025-CS-191
Abdullah Akbar	2025-CS-212
Muhammad Junaid Amir	2025-CS-193
Muhammad Ahmad Chaudhry	2025-CS-184

Submitted to:
Dr. Maimoona Salam

October 31, 2026

Document Control

Project	Hamnawa (ہم نوا) — Accessible Document Reader for Visually Impaired Students
Document	Design Architecture Document: Monolith or Microservices, version 0.1 (draft)
Course	Software Engineering, Department of Computer Science, UET Lahore
Group	Group 2
Supervisor	Dr. Maimoona Salam
Based on	Project Proposal v1.0; Requirements v0.1; System Architecture Document v0.1
Status	Draft for supervisor review

Revision History

Version	Date	Description	Author(s)
0.1	October 31, 2026	First draft of the architecture style decision.	Group 2
0.2	[date]	[After supervisor feedback]	[names]

The decision in one line. Hamnawa is a **modular monolith**: one deployable web application that runs entirely in the user's browser, organised internally into clearly separated modules. It is **not** built as microservices, and it has no backend server of our own.

Contents

1 Introduction **4**

1.1 Purpose 4

1.2 Scope 4

1.3 Method 4

2 Decision Summary **4**

3 Background: Monolith and Microservices **5**

4 Decision Drivers **5**

5 Options Considered **6**

5.1 Option A: Microservices 6

5.2 Option B: Server-Based Monolith 7

5.3 Option C: Client-Side Modular Monolith (chosen) 7

6 Evaluation **7**

6.1 Criteria and Scores 7

6.2 Sensitivity Check 8

6.3 Evidence Behind the Scores 9

7 Why Not Microservices? **9**

8 Keeping the Monolith Modular **10**

8.1 Rules 10

8.2 Source Layout 11

9 Consequences and Trade-offs **11**

10 Evolution Path **12**

11 Build, Test and Deployment **13**

12 Validation Plan and Risks **13**

13 Architecture Decision Record **14**

14 Traceability **15**

A Glossary **15**

1 Introduction

1.1 Purpose

This document records the most basic structural choice of the project: whether Hamnawa should be built as a *monolith* or as a set of *microservices*. It states the decision, the alternatives we weighed, the evidence and reasons for the choice, what the choice costs us, and the conditions under which we would change it.

1.2 Scope

The document covers the *architecture style* and how the code is organised and deployed. The detailed components, use cases, CRC cards and data model are in the System Architecture Document; the requirements are in the Requirements document. Here we only repeat what is needed to justify the decision.

1.3 Method

We followed the architectural design approach of Pressman and Maxim: identify the architectural drivers from the requirements, list candidate styles, evaluate them against the drivers, choose, and record the decision with its consequences (an *architecture decision record*, Section 13).

2 Decision Summary

Decision (ADR-001). Build Hamnawa as a **modular monolith that runs entirely on the client**: a single web application bundle, served as static files, containing all modules (interface, controllers, document processing, speech and storage adapters). Document processing runs in a background *Web Worker* thread inside the same bundle. There is no server application and no microservice.

Three clarifications avoid confusion:

- **The Web Worker is not a microservice.** It is a background thread of the same program, built and shipped in the same bundle; it is not deployed or scaled on its own.
- **The static web host is not a backend.** It only hands out the application files once; it runs none of our logic and stores none of the user's data.
- **The browser's speech engine and the browser vendor's recognition service are third-party services we use**, not services we build or operate.

3 Background: Monolith and Microservices

Aspect	Monolith	Microservices
Definition	One program built, tested and deployed as a single unit.	Many small programs, each owning one business capability, deployed and scaled independently.
Communication	Function calls inside one process.	Network calls (HTTP, messages) between processes.
Data	One shared data model.	Each service owns its own data store.
Deployment	One artifact, one release.	Many artifacts, many pipelines, coordinated versions.
Strengths	Simple to build, test, debug and deploy; fast in-process calls; easy refactoring.	Independent scaling and release; different technologies per service; fault isolation between teams.
Weaknesses	Can become tangled if modules are not kept separate; whole app scales as one.	Heavy operational and testing overhead; network latency and failure; hard for a small team.
Typical fit	Small teams, one product, moderate load, tight latency.	Large organisations, many teams, very high or uneven load.

A **modular monolith** is a monolith whose inside is deliberately divided into modules with strict boundaries and public interfaces. It keeps the simplicity of one deployable unit while preserving most of the design discipline that microservices force on a team.

4 Decision Drivers

The facts of this project that most strongly influence the choice are listed below. Each is traceable to a requirement or constraint.

ID	Driver	What it means for the style	Source
DR1	**Privacy of documents**	Course files should not leave the user's device; no server copy to protect or leak.	NFR-S1, S2, BR-05
DR2	**Zero budget**	No paid servers, databases or cloud services; free static hosting only.	Constraint

ID	Driver	What it means for the style	Source
DR3	**Instant response**	Key presses and voice commands must act within 200 ms to 2 s; every network hop adds delay and a failure point.	NFR-P1 to P4
DR4	**Small team, one semester**	Four people; time for four increments; little capacity to run infrastructure.	Process model
DR5	**Testability and maintainability**	Core logic testable without a browser or microphone; clear module boundaries.	NFR-M1, M2
DR6	**Operational simplicity**	Simple to deploy, roll back and demonstrate; few moving parts that can fail during a demo.	NFR-R1 to R3
DR7	**Load profile**	One user per device; the work is done per user, not shared. There is no concurrent-user load problem to solve.	Product scope
DR8	**Dependence on browser speech**	Speech output and recognition already live in the browser; the application must sit next to them.	NFR-C1 to C3

5 Options Considered

5.1 Option A: Microservices

A backend of independent services behind a gateway, with the browser acting only as a thin client. A typical split would be an upload service, an extraction service, a structure service, a bookmark and settings service and an account service, each with its own data store.

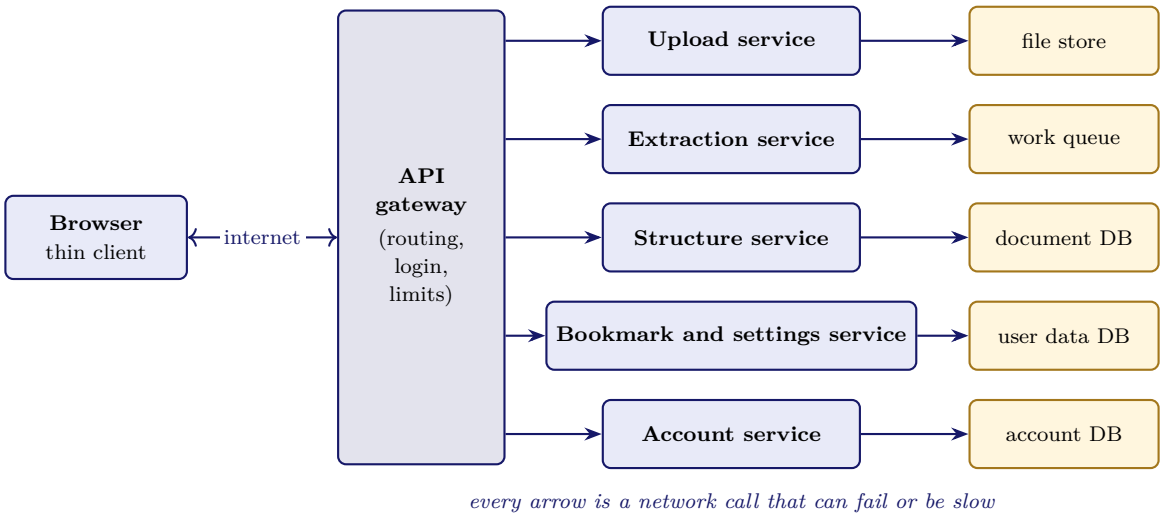


Figure 1: Option A, shown for comparison only: 1 gateway + 5 services + 5 data stores = 11 parts to build, deploy, secure and keep running. The user's documents would travel across the internet to be processed.

5.2 Option B: Server-Based Monolith

One backend application (for example a single web server) that receives documents, extracts and structures them and returns the result. The browser is still a thin client. There is one deployable server, but it needs hosting, storage and security for user files.

5.3 Option C: Client-Side Modular Monolith (chosen)

All logic ships as one bundle that runs in the browser. Heavy parsing runs in a Web Worker. Only the browser's own speech and storage features are used outside our code.

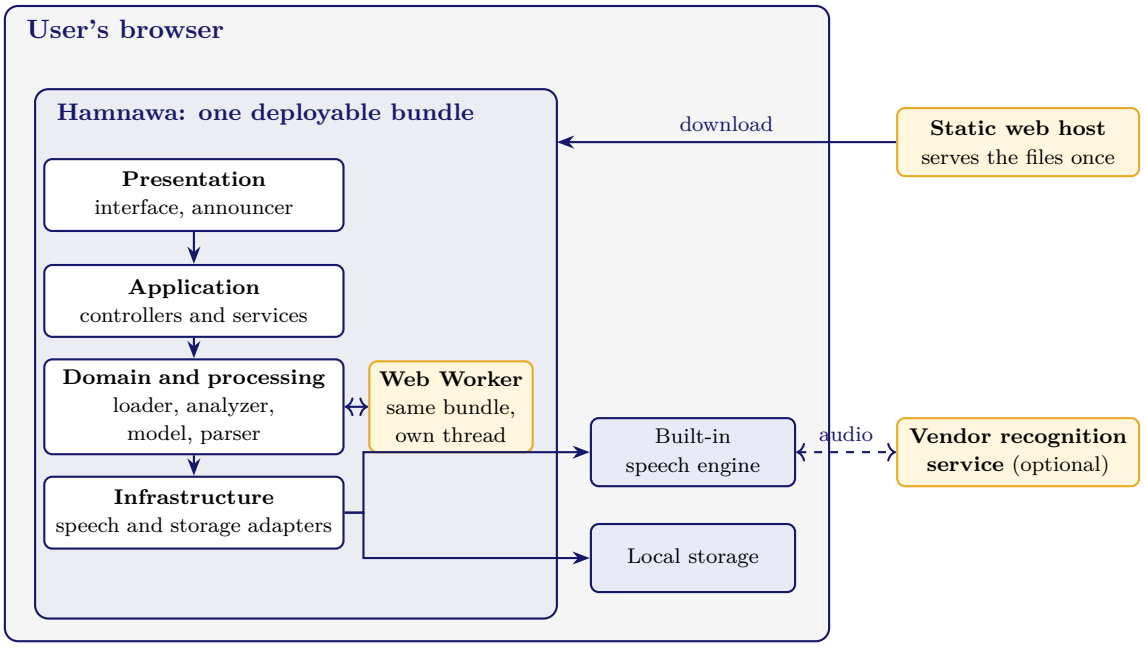


Figure 2: Option C, the chosen design: a single bundle with four internal layers and one background worker. Nothing of ours runs on a server.

6 Evaluation

6.1 Criteria and Scores

Each option was scored from 1 (poor) to 5 (excellent) against the decision drivers. Weights show how much each criterion matters to this project and add up to 100. The scores are the team's judgement, supported by the evidence in Section 6.3.

Criterion	Weight	A: Micro-services	B: Server monolith	C: Client modular monolith
Privacy of documents (DR1)	20	1	2	5
Zero cost (DR2)	15	1	2	5
Responsiveness (DR3)	15	2	3	5
Fit with team and schedule (DR4)	15	1	3	5
Testability and maintainability (DR5)	10	3	3	4
Operational simplicity (DR6)	10	1	3	5
Reliability (DR6, NFR-R)	10	2	3	4
Scalability (DR7)	5	5	3	3
Weighted score (out of 5)	100	**1.65**	**2.65**	**4.70**

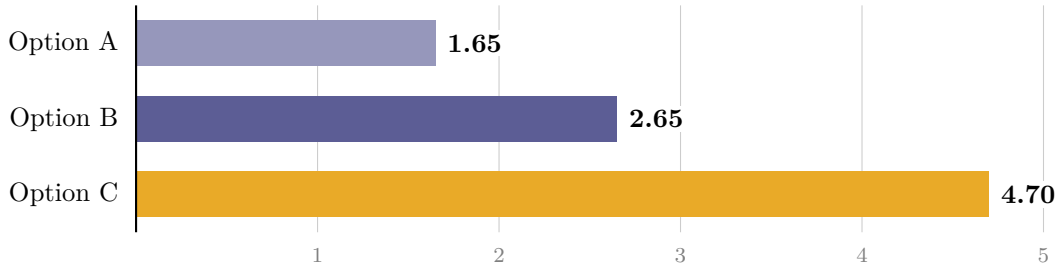


Figure 3: Weighted score of each option (out of 5).

6.2 Sensitivity Check

A decision is more convincing if it survives a change of weights. We repeated the calculation twice:

Scenario	A	B	C
Original weights	1.65	2.65	4.70
All criteria weighted equally	2.00	2.75	4.50
Privacy and cost ignored completely	2.00	3.00	4.54

Option C stays first in every scenario, so the decision does not depend on how much we value privacy and cost.

6.3 Evidence Behind the Scores

Criterion	Reasoning
Privacy of documents	A and B send every document to a server, so we would have to secure, retain and delete user files. C never transmits the file.
Cost	A needs several servers and data stores; B needs at least one always-on server. C needs only free static hosting.
Responsiveness	Each request in A or B adds network time and can fail. In C, commands are function calls on the same device; nothing waits for a server.
Fit with team and schedule	A needs service contracts, a gateway, inter-service testing and deployment for each service. For four students in one semester this would consume the time needed for the product itself.
Testability and maintainability	Services can be tested alone, but A needs contract and end-to-end tests across the network. C keeps the logic in plain modules that unit tests can call directly.
Operational simplicity	A means many things to monitor and debug. C is one bundle, deployed by copying files; a rollback is publishing the previous version.
Reliability	Fewer moving parts means fewer ways to fail during a demo or a lecture. A single user's device is the only place where processing happens.
Scalability	A scales best by design. C scales naturally because each user brings their own computing power, but a very large file is limited by one device.

7 Why Not Microservices?

Microservices are a good solution to problems that Hamnawa does not have. The reasons we rejected them are specific:

1. **There is no scale problem to solve.** Each user processes their own documents on their own device. There is no shared load to split across servers or scale independently (DR7).
2. **They would break our privacy promise.** To process documents in a service, the file must be uploaded. That defeats NFR-S1 and BR-05 and creates stored personal data we would then have to protect (DR1).
3. **They would cost money we do not have.** Even at small scale, several services and data stores need hosting, and always-on hosting is not free (DR2).
4. **They would slow the experience for the people who need it most.** A blind student relies on instant audio feedback. Network round trips and cold starts add delay and silent failures (DR3, DR6).

5. **The team is too small.** Microservices solve *organisational* problems (many teams working independently). With four members there is no coordination problem to solve, only an overhead to pay (DR4).
6. **The domain is one tightly connected flow.** Open, extract, structure, read and navigate all depend on the same document model. Splitting it across services would create chatty, fragile calls for no benefit.
7. **More parts mean more ways to fail.** Distributed systems add partial failure, version mismatch and network problems. Our reliability requirement says a bad file must never leave the user in silence (NFR-R1).

Why not a server-based monolith (Option B)? It is simpler than A and would be acceptable, but it still needs hosting and sends documents off the device, so it scores below C on privacy, cost and responsiveness without any compensating benefit for this product.

8 Keeping the Monolith Modular

A monolith is only a good choice if it stays organised. We therefore treat it as a *modular monolith* and set rules that give us the discipline of services without the network.

8.1 Rules

1. **Layering.** A layer may use layers below it and never a layer above it. Information flows upward only through events.
2. **Public interfaces only.** A module is used only through its documented interface. No module reads another module's internal data.
3. **Adapters at the edge.** Browser features (speech, storage, workers) are reached only through adapter classes.
4. **One data owner.** Each kind of stored data is owned by one module (for example bookmarks by the bookmark manager); others ask for it.
5. **Replaceable extraction.** The document extractor sits behind an interface, so it can later be moved to a server without changing its callers.
6. **Enforced by tools.** A lint rule and a continuous-integration check fail the build if code imports from a higher layer.

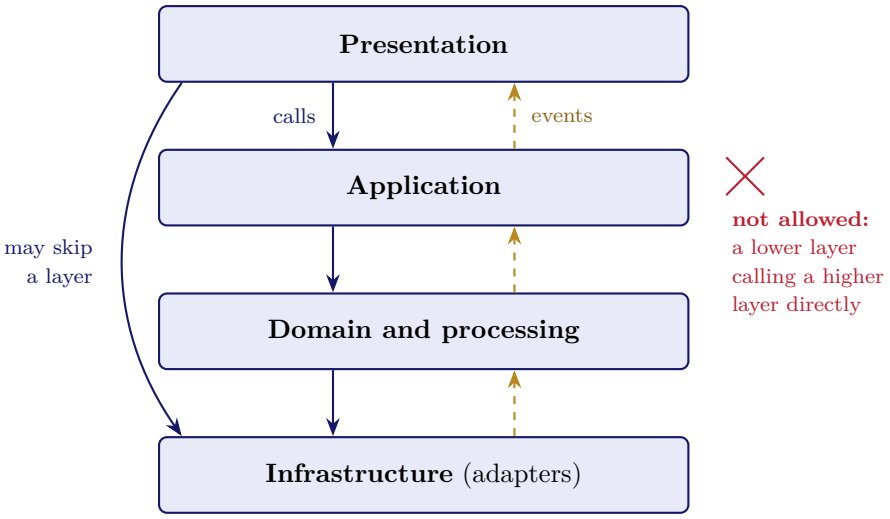


Figure 4: Dependency rules. Solid arrows are allowed calls (downward, skipping layers is fine). Dashed arrows are events travelling upward. A direct upward call is forbidden.

8.2 Source Layout

The folder structure mirrors the layers, so a misplaced dependency is easy to see in review.

```
hamnawa/
  src/
    ui/          UIController, Announcer, views
    app/         Playback, Navigation, VoiceCommand controllers,
                 BookmarkManager, ResumeService, SettingsManager
    domain/      DocumentLoader, StructureAnalyzer, DocumentModel,
                 CommandParser
      extract/ DocumentExtractor (pdf/, pptx/)   runs in the Web Worker
    infra/       SpeechSynthesizerAdapter, SpeechRecognizerAdapter,
                 StorageService
    shared/      event definitions, types, utilities
  tests/         unit and integration tests, mirroring src/
  docs/          proposal, requirements, architecture, design
```

9 Consequences and Trade-offs

Consequence	Effect	Mitigation
Good: documents stay on the device	Strong privacy; nothing to secure or delete on a server.	—
Good: zero running cost	Free static hosting; nothing to pay for during or after the course.	—

Consequence	Effect	Mitigation
Good: fast, simple, testable	Direct calls; one build; plain unit tests; easy demo.	—
Bad: performance depends on the user's device	A very large or complex file may be slow on an old laptop.	Web Worker; file limits (BR-01); early performance test.
Bad: no cross-device sync or accounts	Bookmarks stay in one browser.	Out of scope; see Section 10 for a future sync service.
Bad: no central usage data	We cannot measure real use remotely.	Gather feedback through interviews and testing sessions.
Bad: browser dependence	Features differ by browser.	Adapters with feature detection; keyboard always works.
Bad: risk of a tangled monolith	Modules could slowly become entangled.	Rules, folder layout and automated checks above.

10 Evolution Path

The decision is right *for the current scope*. It is not permanent. The table shows what could change and which seam in the design lets us move that piece out of the monolith without a rewrite.

Trigger	Possible new service	Seam already in the design
Scanned documents (OCR) or heavy parsing	Server-side extraction service	`DocumentExtractor` interface; format-independent document model.
Accounts and cross-device bookmarks	Sync service with a user database	`StorageService` adapter.
Automatic image or equation descriptions	Description service (AI)	Content blocks of kind *notice* for images and tables.
Institution-wide deployment with shared libraries	Document library service	Document fingerprint as a stable identifier.
Usage analytics for improvement	Telemetry service (with consent)	Event bus; no module depends on it.

Each of these would be introduced as a *single* extracted service behind an existing interface, not

as a wholesale move to microservices.

11 Build, Test and Deployment

Because there is one artifact, the delivery pipeline is short.

1. **Commit** to a branch of the Git repository; each change is reviewed by a second member.
2. **Lint and layer check:** style rules and the dependency rule from Section 7.
3. **Unit and integration tests:** core logic runs without a browser; the pipeline runs on the sample document set.
4. **Build** one static bundle.
5. **Accessibility checks:** automated checks plus a manual run with the NVDA screen reader before each increment is accepted.
6. **Deploy** the bundle to free static hosting (a preview address first, then the main address).
7. **Rollback:** publish the previous bundle; no data migration is needed because user data lives in the user's own browser and settings are versioned.

Environment	Where it runs	Purpose
Development	Each member's laptop	Daily work, local tests.
Preview	Free static hosting (temporary address)	Review of each increment by the team and supervisor.
Production / demo	Free static hosting; local copy as backup	Final demonstration; local copy protects against hosting or network failure.

12 Validation Plan and Risks

The decision rests on one assumption: *a browser can process a typical course document fast enough.* We will test it early instead of trusting it.

ID	Risk or check	Action	When
V1	Parsing a 100-page PDF in the browser may exceed the 20 s target (NFR-P4).	Performance spike in a Web Worker with real course files; measure time and memory.	Increment 1
V2	Reading order quality may be too poor on complex layouts without server tools.	Test corpus; compare results; document limits.	Increment 1

ID	Risk or check	Action	When
V3	Modules may drift apart from the layer rules.	Automated layer check from the first commit.	Increment 1 onward
V4	Browser speech differences may break behaviour.	Adapter tests on Chrome and Edge; clear fallback message.	Increment 1 and 3

Fallback plan. If V1 shows the browser is too slow, we will introduce *one* small extraction service (the first row of the evolution table) and keep everything else unchanged. This would be recorded as a new decision record that supersedes ADR-001 in part.

13 Architecture Decision Record

ID and title	**ADR-001: Modular monolith running entirely on the client**
Status	Accepted (draft v0.1), to be reviewed with the supervisor.
Date	October 31, 2026
Context	Hamnawa helps visually impaired students study from course documents. Requirements demand privacy of documents, instant response, zero cost and delivery by a four-member team in one semester. Each user works alone on their own device.
Decision	Build one web application bundle containing all modules, running in the browser, with document processing in a Web Worker. No server application and no microservices. Keep the code modular with enforced layer rules.
Alternatives considered	A: microservices; B: server-based monolith; C: client-side modular monolith (chosen). Scores: see Section 6.
Consequences	Strong privacy, no cost, fast response, simple delivery. Performance depends on the user's device; no sync across devices; no central usage data. Mitigations in Section 9.
Review triggers	Performance spike V1 fails; a requirement for accounts, sync, OCR or shared libraries is accepted; the team or user base grows substantially.
Related	System Architecture Document: decisions D1 (browser-only processing), D2 (layers and pipeline), D3 (adapters).

14 Traceability

ID	Driver	How the decision answers it	Requirements
DR1	Privacy	Processing and storage stay on the device.	NFR-S1, S2
DR2	Zero budget	Static hosting, free libraries, no servers.	Cost constraint
DR3	Instant response	In-process calls; background worker for heavy work.	NFR-P1 to P4
DR4	Small team	One codebase, one pipeline, one deployment.	Process model
DR5	Testability	Plain modules and adapters that can be faked.	NFR-M1, M2
DR6	Operational simplicity	One artifact, easy rollback, local backup demo.	NFR-R1 to R3
DR7	Load profile	Each user's device does its own work.	Product scope
DR8	Browser speech	Application sits beside the browser's speech engine through adapters.	NFR-C1 to C3, FR-39

A Glossary

Monolith An application built, tested and deployed as one unit.

Modular monolith A monolith divided into modules with strict boundaries and public interfaces.

Microservices An application made of many small, independently deployed services that talk over a network.

API gateway A single entry point that routes requests to services.

Web Worker A browser feature that runs code in a background thread.

Adapter A class that hides a browser or library feature behind an interface of our own.

Static hosting A server that only hands out files and runs no application logic.

ADR Architecture decision record: a short document of a decision, its context and consequences.

Seam A place in the design where a piece can be replaced or moved without changing its callers.